Software Design Specifications

For

Peer-to-Peer Mentoring Platform - Version 1.0

Prepared by: Team 5

Ayush: Team Lead and System Architect
Amulya: UI/UX and Accessibility Engineer
Hrutika: Frontend / React and Design Engineer
Amrutha: Backend and Booking Interface Engineer
Amitha: Database and System Design Engineer
Rusheel: Authentication and User Management Engineer

Document Information

Title: Peer-to-Peer Mentoring Platform

Document Version No: Version 1.0

Project Manager: Ayush Garg

Document Version Date: 5/04/2026

Prepared By: Amitha Gundapaneni

Preparation Date: 6/04/2026

Version History

Ver. No.	Ver. Date	Revised By	Description	Filename
Version 1.0	6/04/2026	Amitha Gundapaneni	Design Overview of the System	SDS_Team5_Version1.0

Table of Contents

1 Introduction

The Software Design Specification (SDS) explains the system design of the Peer-to-Peer Mentoring Platform. It provides an overview of how the system works, what features it has, and how its different parts are connected. This document is mainly useful for understanding the application's structure and development.

1.1 Purpose

The purpose of this document is to describe how the Peer-to-Peer Mentoring Platform is designed. It explains the main features of the system and its implementation. This document can be used by development teams, related stakeholders, and others interested in understanding how the system works.

1.2 Scope

The system is a web-based platform designed to help students connect with peer mentors for academic support. It includes features such as user authentication, session booking, doubt discussions, mentor matching, messaging, and personalised recommendations. The scope of this SDS is to describe the internal design and structure of these features, including how different components interact and function within the system.

1.3 Definitions, Acronyms, and Abbreviations

- SDS – Software Design Specification
- UI/UX – User Interface / User Experience
- API – Application Programming Interface
- DB – Database
- Auth – Authentication

1.4 References

- IEEE guidelines for Software Design
- Class notes and materials
- Online sources used for understanding system design
- SRS documentation

2 Use Case View

This section explains how users will interact with the platform and what actions they can perform with each functionality feature. It includes the main features that users will use in the system.

2.1 Use Case

User Registration

- The user provides the required personal details
- The system validates and stores the information (including email verification)
- A new user account is created

User Authentication (Login)

- The user enters valid credentials
- The system verifies the credentials
- Access to the system is granted upon successful validation

Profile Management

- The user updates profile information
- The system stores updated details
- Mentors can specify expertise and availability
- Users can personalise their learning pathway and view recent activity based on their subject-value requirement from the platform

Session Scheduling and Booking

- The student views available mentor time slots
- The student selects a preferred slot
- The student can choose to book a group session
- The system confirms and records the booking
- Confirmation notifications are sent to both the user and the mentor

Doubt Posting

- The user submits a doubt through the interface
- The system processes and stores the doubt
- The system may suggest similar existing doubts
- Unique doubts are posted on the forum

Doubt Resolution

- Mentors or faculty respond to posted doubts
- The system displays responses to users
- Interaction is stored for future reference

Messaging and Announcements

- Users exchange messages with mentors through the system, facilitating session-related discussions
- Mentors post announcements
- The system delivers notifications to relevant users

Recommendation System

- The system analyses user data and activity
- Relevant mentors or content are recommended
- Users can accept or ignore recommendations

Administrative Management

- The administrator manages user accounts
- The system enables monitoring of platform activity
- Content moderation actions can be performed

3 Design Overview

This section gives an overview of the overall design of the Peer-to-Peer Mentoring Platform. It shows how the system is organised at a high level and how different parts are connected. The design follows the system requirements and includes major components like user interface, authentication, reminders, notifications, and emergency features. It also shows how these modules are divided and work together to form the complete system.

3.1 Design Goals and Constraints

Goals:
Modular and scalable system
Fast response for real-time features (chat, booking)
Easy-to-use UI
Efficient mentor matching using ML

Constraints:
Web-based system (browser-dependent)
Limited development time
Dependence on external (third-party) services (DB, ML, notifications)

3.2 Design Assumptions

- Users have internet access
- System runs on standard browsers
- Database is always available
- ML module provides reasonably accurate outputs

3.3 Significant Design Packages

The system is divided into different sections:

- **Authentication Module** – login, registration
- **User Profile Module** – manage user data
- **Session Management Module** – booking, scheduling
- **Doubt System Module** – posting, answering
- **ML Module** – recommendations, tagging, matching
- **Messaging Module** – chat, announcements
- **Admin Module** – moderation and control

3.4 Dependent External Interfaces

External System	**Used By**	**Purpose**
Database (Supabase)	All modules	Store user, sessions, doubts
ML Service	ML Module	Recommendations & tagging
Notification Service	Session/Doubt modules	Alerts & reminders
Payment Gateway (Omitted for Pilot Phase only)	Booking module	Session payments
Web-Application Host (Vercel)	All Modules	To host the application, making it available for global use

3.5 Implemented Application External Interfaces (and SOA web services)

The table below lists the implementation of public interfaces that this design makes available for other applications.

Interface	**Module**	**Function**
Auth API	Authentication	Login/Register
Booking API	Session Module	Book sessions
Doubt API	Doubt Module	Post/view doubts
Chat API	Messaging Module	Real-time messaging
Recommendation API	ML Module	Generate suggestions

4 Logical View

The logical view describes the detailed design of the system using different layers.

- The top layer is the **User Interface layer**, where users interact with the system to perform actions such as login, booking mentoring sessions, posting doubts, and communicating with mentors.
- The next layer is the **Application layer**, which handles the core logic of the system, including processing user inputs, managing session bookings, handling doubt posting and resolution, and coordinating user activities.
- Below that is the **Service layer**, which manages communication between different modules such as authentication, session management, messaging, and the recommendation system, ensuring smooth interaction across components.
- The bottom layer is the **Database layer**, where all system data, such as user profiles, sessions, doubts, messages, and recommendations, are stored and managed.

These layers work together to fulfil the system's functionalities. Each module collaborates with others to support features like registration, session booking, doubt resolution, messaging, and personalised recommendations. The system design can be further modularised to clearly define individual components and their responsibilities.

4.1 Design Model

User Class

- Stores user details such as user ID, name, email, and role
- Handles user-related operations like login and profile updates

Mentor Class

- Extends the User class
- Stores mentor-specific attributes such as expertise and availability
- Manages mentoring sessions and availability

Session Class

- Stores session details such as session ID, mentor, student, and time
- Handles session booking, scheduling, and cancellation

Doubt Class

- Stores doubt-related data such as content, tags, and user ID
- Handles posting, searching, and managing doubts

Response Class

- Stores responses to doubts
- Links mentors or faculty to posted doubts

Message Class

- Handles communication between users
- Manages sending and receiving messages

Recommendation Class

- Generates personalised recommendations using user data
- Suggests mentors, sessions, or relevant content.

4.2 Use Case Realisation

This section describes how the major use cases of the system are implemented through interaction between different components and modules.

For User Login:

- User enters login details in the UI
- The Authentication module verifies the credentials
- Access is granted to the system

For Session Booking:

- Student selects mentor and available time slot
- Session module processes the booking
- Booking is confirmed and stored

For Doubt Posting:

- User enters doubt details in the UI
- Doubt module processes and stores the input
- The system may suggest similar doubts if existing

For Doubt Resolution:

- Mentor or faculty views the posted doubt
- Response is submitted through the system
- The system displays the response to users
- The thread is opened for follow-up questions/comments

For Messaging:

- User sends a message through the UI
- Messaging module processes the message
- Message is delivered to the recipient

For Recommendation System:

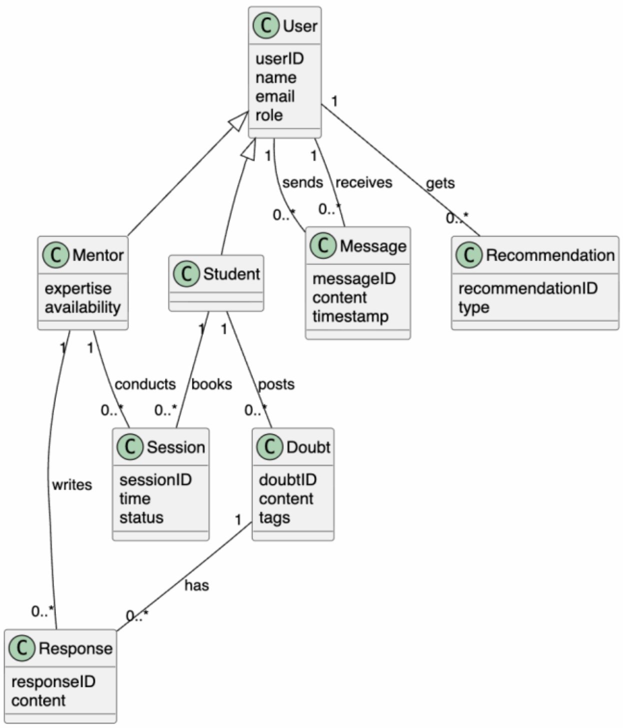
- System analyses user data and activity
- Recommendation module generates suggestions
- Relevant mentors or content are displayed

At a higher level, modules interact with each other to complete these actions. At a lower level, different classes inside each module work together to perform specific tasks.

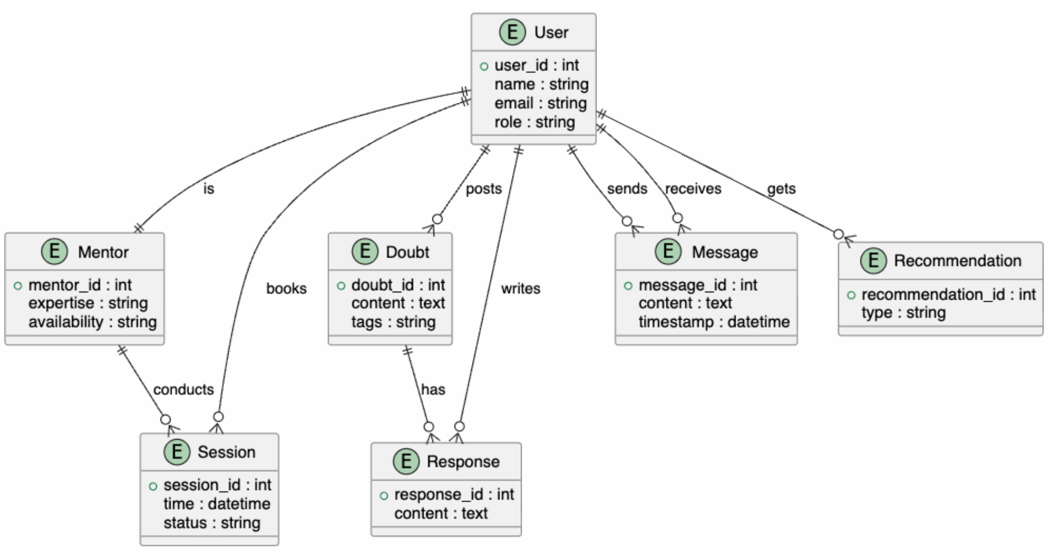
5 Data View

This section describes how data is stored and managed in the system. It explains the structure of data and how different data elements are related.

5.1 Domain Model



5.2 Data Model (persistent data view)



5.2.1 Data Dictionary

User Table → stores user details
Mentor Table → stores mentor-specific information such as expertise and availability
Session Table → stores session booking and scheduling information
Doubt Table → stores doubts posted by users
Response Table → stores responses to posted doubts
Message Table → stores messages exchanged between users
Recommendation Table → stores personalised recommendations for users

6 Exception Handling

The system handles errors properly to ensure smooth operation and avoid failures.

- If login credentials are incorrect, an error message is displayed
- If session booking fails due to unavailability, the user is prompted to choose another time slot
- If the submission fails, the user is asked to retry
- If message delivery fails, the system attempts to resend or notifies the user
- If the recommendation system fails, default suggestions are provided
- If the database connection fails, the system shows an error and retries the operation

7 Configurable Parameters

Configuration Parameter Name	**Definition and Usage**	**Dynamic**
Session Duration	Time set for each mentoring session	Yes
Notification Settings	Controls alerts for sessions, messages, and updates	Yes
Mentor Availability	Defines available time slots set by mentors	Yes
Recommendation Preferences	Controls how recommendations are generated for users	Yes
User Profile Settings	Allows users to update personal and account preferences	Yes

8 Quality of Service

This section describes how the system maintains availability, security, performance, and monitoring during its usage.

8.1 Availability

The application is designed to be available for users at all possible times. It is intended to work properly whenever users need to set reminders or use emergency features.

- The system supports continuous usage with minimal downtime
- Regular updates or maintenance may require short downtime
- Data loading and updates are handled in a way that does not affect users much

8.2 Security and Authorisation

The system ensures that user data is secure and only authorised users can access the application.

- Users must log in using their username and password
- Email authentication is used to verify user identity
- Authorisation ensures users can only access their own data
- Sensitive data, such as personal details and messages, is protected
- No additional personal data of the user is displayed during session booking or doubt posting. Only the required information (such as name or username) will be displayed
- User access can be managed based on roles (Student, Mentor, Admin)

8.3 Load and Performance Implications

The system is designed to handle normal user load efficiently.

- The system can handle multiple users accessing it at the same time
- Session booking, messaging, and doubt processing are performed quickly
- Data is stored efficiently to avoid delays
- Performance is maintained even as the number of users increases

8.4 Monitoring and Control

The system includes monitoring features to track its performance and usage.

- The system monitors session activities, messaging, and doubt processing
- Errors and failures are tracked and handled
- Basic logs are maintained for system activities
- The system ensures smooth functioning by controlling different processes